

Clasificación de variedades de arroz mediante regresión logística y Random Forest

Marco Iván Flores Villanueva
Instituto Tecnológico y de Estudios Superiores de Monterrey
Querétaro, México
A01276586@tec.mx

Abstract — Este documento presenta la implementación de dos modelos de aprendizaje supervisado para clasificar granos de arroz de las variedades Cammeo y Osmancik a partir de siete características físicas. Primero se desarrolló un modelo de regresión logística a mano mediante la función sigmoide, cross-entropy y gradiente descendente. Después, se implementó un Random Forest con la librería scikit-learn y se analizaron sus resultados antes y después de un ajuste de hiperparámetros. El dataset se dividió de manera estratificada en 60% para entrenamiento, 20% para validación y 20% para prueba. La regresión logística alcanzó una precisión de 92.65% en entrenamiento, 93.57% en validación y 92.65% en prueba. El Random Forest con parámetros por default obtuvo 100% en entrenamiento y 92.65% en validación, lo cual indicaba sobreajuste. Después de buscar hiperparámetros acertados con GridSearchCV, la brecha entre entrenamiento y validación disminuyó de 7.35 a 0.66 puntos porcentuales. El modelo ajustado obtuvo 93.96% en entrenamiento, 93.31% en validación y 92.65% en prueba, lo cual indica un sesgo bajo, una varianza baja y un fitting adecuado.

Keywords — *rice, logistic regression, Random Forest, hyperparameter tuning, bias, variance, overfitting, sigmoid, cross-entropy, gradient descent, Turkey*

I. INTRODUCCIÓN

Los algoritmos de machine learning permiten construir modelos capaces de aprender relaciones entre variables de entrada y una variable objetivo a partir de ejemplos que los entrenan. En ese ámbito, los problemas de clasificación buscan asignar cada observación a una categoría del dataset, y cuando sólo hay 2 clases, la regresión logística y Random Forest son unas de las técnicas más utilizadas por su simplicidad y facilidad de comprensión.

Para este trabajo, decidí utilizar el dataset Rice - Cammeo and Osmancik, compuesto por 3810 granos de arroz clasificados en 2 variedades. A partir de las imágenes procesadas de los granos, se identificaron siete características físicas: área, perímetro, longitud del eje mayor, longitud del eje menor, excentricidad, área convexa y extensión. Estas características permiten analizar diferencias de tamaño y forma entre ambas variedades, por lo cual resulta interesante averiguar si también son suficientes para categorizarlas completamente y poder predecir qué variedad de arroz corresponde a cada grano.

Es por esto que el objetivo de este trabajo es implementar y evaluar dos modelos de aprendizaje supervisado para clasificar granos de arroz de las 2 variedades dadas sus características físicas. El primer modelo consiste en una regresión logística implementada manualmente con numpy, mientras que el segundo utiliza el algoritmo de Random Forest con scikit-learn. En el caso de este último, también se analiza su desempeño mediante conjuntos de entrenamiento, validación y prueba, se diagnostica su sesgo, varianza y fitting, y se realiza una búsqueda de hiperparámetros para mejorar su precisión.

II. DATASET Y EDA

A. Descripción del dataset

El dataset Rice (Cammeo and Osmancik), publicado en el UCI Machine Learning Repository en 2019, contiene 3810 observaciones correspondientes a granos de arroz de las variedades Cammeo y Osmancik. Sus datos fueron obtenidos mediante el procesamiento de imágenes de los granos, y se extrajeron siete características: *Area*, *Perimeter*, *Major Axis Length*, *Minor Axis Length*, *Eccentricity*, *Convex Area* y *Extent*. Mientras que estas variables describen las propiedades físicas de cada grano, la variable objetivo *Class* clasifica a cada observación como Cammeo u Osmancik, y es debido a eso que este dataset representa un problema de clasificación binaria que puede ser atacado mediante algoritmos como la regresión logística y Random Forest.

Variable Name	Role	Type	Description	Units
Area	Feature	Integer	Returns the number of pixels within the boundaries of the rice grain	px
Perimeter	Feature	Continuous	Calculates the circumference by calculating the distance between pixels around the boundaries of the rice grain	px
Major_Axis_Length	Feature	Continuous	The longest line that can be drawn on the rice grain, i.e. the main axis distance, gives	
Minor_Axis_Length	Feature	Continuous	The shortest line that can be drawn on the rice grain, i.e. the small axis distance, gives	
Eccentricity	Feature	Continuous	It measures how round the ellipse, which has the same moments as the rice grain, is	
Convex_Area	Feature	Integer	Returns the pixel count of the smallest convex shell of the region formed by the rice grain	
Extent	Feature	Continuous	Returns the ratio of the region formed by the rice grain to the bounding box	
Class	Target	Binary	Cammeo and Osmancik	

B. Revisión de los datos y limpieza

Para tener un mejor panorama de los datos antes de construir cualquier modelo, se aplicó una revisión inicial de su estructura y calidad. El dataset importado era de 3810 observaciones y 8 columnas correspondientes a las 7 características físicas y a la clase. Fue ahí que se comprobó que las variables eran efectivamente numéricas, y que *Class* era un string.

```
RangeIndex: 3810 entries, 0 to 3809
Data columns (total 8 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Area                3810 non-null   int64
1   Perimeter           3810 non-null   float64
2   Major Axis Length   3810 non-null   float64
3   Minor Axis Length   3810 non-null   float64
4   Eccentricity         3810 non-null   float64
5   Convex Area         3810 non-null   int64
6   Extent              3810 non-null   float64
7   Class               3810 non-null   str
```

Después se revisó si existían valores faltantes o registros duplicados, y por fortuna no era el caso, por lo que no fue necesario realizar ninguna imputación o eliminación de observaciones.

```
Valores faltantes:
Area          0
Perimeter     0
Major Axis Length  0
Minor Axis Length  0
Eccentricity   0
Convex Area    0
Extent         0
Class          0
dtype: int64

Filas duplicadas:
0
```

C. Distribución de clases

La distribución de Class fue de 2180 observaciones de Osmancik y 1630 de Cammeo, aproximadamente al 57.22% y 42.78% del dataset respectivamente. Aunque existe una mayor cantidad de observaciones de Osmancik, se consideró que ambas clases contaban con una representación suficiente dentro del conjunto.

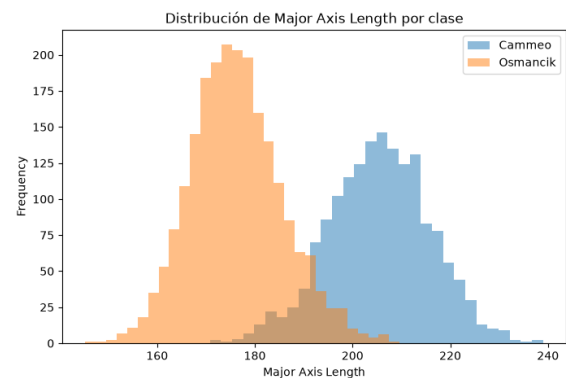
```
Clases existentes:
<StringArray>
['Cammeo', 'Osmancik']
Length: 2, dtype: str

Distribución de clases:
Class
Osmancik    2180
Cammeo      1630
Name: count, dtype: int64

Distribución porcentual de clases:
Class
Osmancik    57.217848
Cammeo      42.782152
Name: proportion, dtype: float64
```

D. Análisis de las características físicas

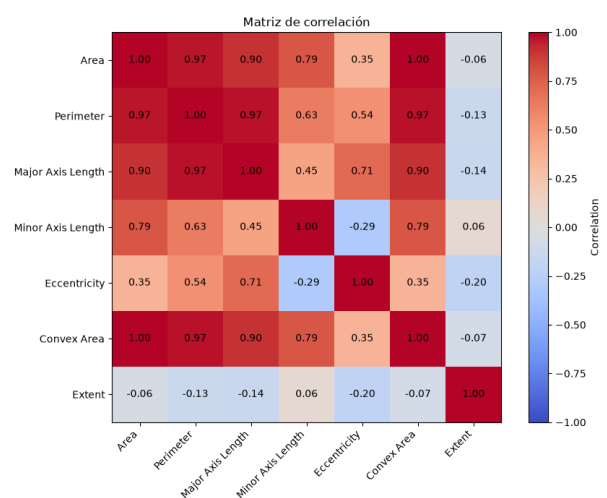
Para determinar si las características físicas presentaban diferencias entre las dos clases, se calcularon medidas descriptivas para cada variedad y se compararon sus distribuciones. Los resultados mostraron diferencias suficientemente visibles en variables relacionadas con el tamaño del grano, por ejemplo, el área promedio siendo de aproximadamente 14163 para Cammeo y 11550 para Osmancik. De igual forma, Cammeo presentó mayores valores promedio de perímetro, longitud del eje mayor y área convexa. También se observaron diferencias en características relacionadas con la forma. La excentricidad promedio fue mayor en Cammeo, mientras que Extent mostró un promedio ligeramente mayor para Osmancik. Estas diferencias se pueden interpretar como que las variables sí contienen información útil para distinguir ambas variedades.



En esta figura se puede observar esta diferencia para la variable Major Axis Length. Aunque hay una región donde las dos clases se superponen, la distribución de Cammeo se encuentra generalmente hacia valores mayores, mientras que la de Osmancik hacia valores menores. El hecho de que haya superposición también refuerza la interpretación de que una sola característica puede no ser suficiente para clasificar correctamente todas las observaciones, y justifica el uso de múltiples variables para la regresión.

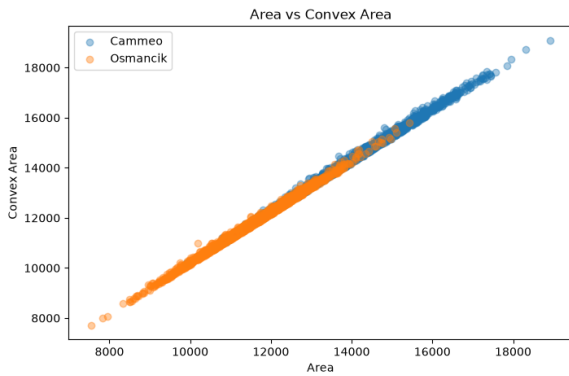
E. Relaciones entre las variables

Finalmente, se calculó la matriz de correlación entre las siete características para poder identificar relaciones entre ellas. Se encontraron correlaciones positivas altas entre varias variables relacionadas con el tamaño del grano. Area y Convex Area presentaron una correlación de 0.999, mientras que Perimeter y Major Axis Length presentaron una correlación de 0.972. Asimismo, Area y Perimeter mostraron una correlación de 0.966. Sin embargo, Extent presentó correlaciones relativamente bajas con las demás características. Esto puede explicarse porque a diferencia de otras variables que miden directamente dimensiones o tamaño del grano, Extent representa la proporción entre el área ocupada por el grano y el área de su bounding box. Aunque esta característica describía un aspecto diferente de las dimensiones del grano, podía aportar información complementaria al modelo y no se descartó.

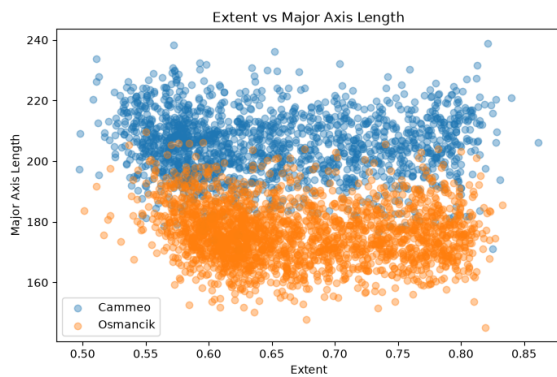


Podemos observar en la siguiente figura que la relación entre Area y Convex Area es lineal, pues su correlación es de 1, y conforme aumenta el área del grano, su área convexa también lo hace en la misma proporción. La gráfica muestra una separación parcial entre las variedades, pues Osmancik se

concentra mayormente en valores menores y Cammeo en valores mayores.



Otra gráfica interesante es la de Extent contra Major Axis Length, pues esta no presenta una relación lineal evidente. Para valores similares de Extent pueden encontrarse granos con longitudes de eje mayor considerablemente diferentes. Esto confirma el hecho de que Extent captura un aspecto distinto de la geometría del grano, y puede aportar información complementaria al utilizarse junto a las demás variables.



III. PREPARACIÓN DE DATOS E IMPLEMENTACIÓN MANUAL

A. Preparación de datos

Para aplicar la regresión logística fue necesario primero transformar la variable objetivo Class, debido a que sus valores originales eran las etiquetas de texto “Cammeo” y “Osmancik”. Se asignó el valor 0 a Osmancik y el valor 1 a Cammeo, obteniendo así una variable objetivo binaria compatible con la función sigmoide.

De igual forma, el dataset original se encontraba ordenado de acuerdo con la variedad de arroz, es decir, primero las observaciones de Cammeo y después las de Osmancik. Es por eso que se implementó manualmente una división estratificada con una seed aleatoria para hacer shuffle y que el orden fuera igual en otras ejecuciones. Se asignó el 60% de las observaciones al conjunto de entrenamiento, 20% al conjunto de validación y 20% al conjunto de prueba, y gracias a la estratificación, se logró conservar aproximadamente la misma proporción de Cammeo y Osmancik en estos tres conjuntos.

Por último, y debido a que las características presentaban magnitudes diferentes, se aplicó estandarización mediante z-score. En un inicio se había utilizado mean scaling para reducir estas diferencias de magnitud, pero para la versión final se decidió sustituir por estandarización mediante z-score,

ya que esta transformación permite centrar cada característica alrededor de cero y expresarla en unidades de desviación estándar.

$$z = \frac{x - \mu}{\sigma}$$

μ = Mean

σ = Standard Deviation

Fórmula de z-score.

B. Modelo de regresión logística

Para realizar la clasificación se implementó un modelo de regresión logística usando únicamente la librería numpy para las operaciones matemáticas. El modelo calcula primero una combinación lineal de las siete características de entrada mediante un vector de pesos w y un bias b .

Debido a que esta combinación puede producir cualquier valor real, su resultado se transforma mediante la función sigmoide, la cual convierte valores a un rango entre 0 y 1 y se interpreta como la probabilidad de que una observación pertenezca a la clase positiva. Debido a la codificación de clases que se usó, esa salida representa la probabilidad de que un grano pertenezca a la variedad Cammeo.

C. Función de costo y entrenamiento

En un inicio se le asignan pesos en ceros al modelo, al igual que el bias, y es gracias a la gradiente descendente y a cross entropy que estos cambian. Como el modelo empieza en blanco, su primera predicción es mala e "ignorante", por lo que se realiza una medición del error producido entre su estimación con la realidad con la función de cross entropy:

$$H(p, q) = - \sum_i p_i \log q_i = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

En esta función, m representa el número de observaciones, y_i la clase real y_{hat} la probabilidad estimada por el modelo. En el código, la implementación se realizó con operaciones de numpy. De igual forma, para evitar valores indefinidos producidos por $\log(0)$, las probabilidades se limitaron a un intervalo ligeramente mayor a cero e inferior a uno antes de calcular la función de costo.

```
219 # Función para calcular el costo mediante cross entropy
220 def binary_cross_entropy(y, y_hat):
221     epsilon = 1e-15
222     # Se evitan valores iguales a 0 o 1 para no calcular log(0)
223     y_hat = np.clip(y_hat, epsilon, 1 - epsilon)
224     return -np.mean(y * np.log(y_hat) + (1 - y) * np.log(1 - y_hat))
```

En cada epoch se calcularon las probabilidades de todas las observaciones de entrenamiento y después la diferencia entre las predicciones y sus valores reales. A partir de este error se calcularon los gradientes correspondientes a los pesos y al bias.

```
247 # 3. Gradientes
248 error = y_hat - y
249
250 dw = (1 / n_samples) * np.dot(X.T, error)
251 db = (1 / n_samples) * np.sum(error)
```

En el código, el vector dw representa el gradiente del costo respecto a los pesos, y db representa el gradiente correspondiente al bias.

Finalmente, los parámetros se actualizan en dirección contraria al gradiente con el objetivo de reducir progresivamente la función de costo.

```
253 # 4. Actualización de parámetros
254 weights = weights - learning_rate * dw
255 bias = bias - learning_rate * db
```

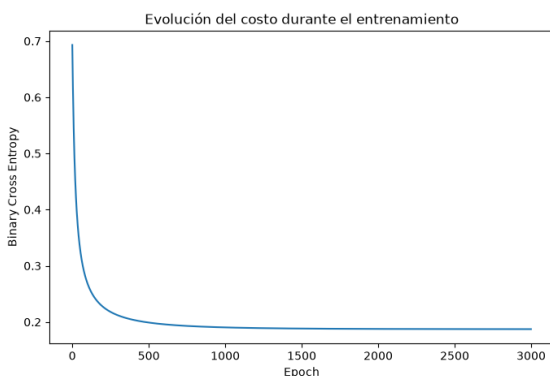
D. Configuración del entrenamiento

Este modelo manual fue entrenado mediante gradiente descendente por batch, utilizando un learning rate de 0.03 durante 3000 epochs, y con los pesos y el bias inicializados en cero. En cada epoch se calcularon las probabilidades, la binary cross-entropy y los gradientes correspondientes a los pesos y al intercepto. Para convertir las probabilidades en clases se usó un umbral de 0.5, haciendo que los valores mayores o iguales se clasificaran como Cammeo y los menores como Osmancik. Al finalizar el entrenamiento, el modelo se evaluó por separado en los conjuntos de entrenamiento, validación y prueba.

IV. RESULTADOS DE LA IMPLEMENTACIÓN MANUAL

A. Convergencia del modelo

Durante el entrenamiento se registró el valor de la función de costo en cada epoch para así poder observar el comportamiento del gradiente descendente. El modelo comenzó con un costo de aproximadamente 0.693, correspondiente a las probabilidades iniciales generadas con pesos iguales a cero, y presentó una disminución a lo largo de las 3000 epochs y conforme avanzaba el gradiente descendente. Alrededor de la epoch 2900, el costo había disminuido hasta 0.187.



Epoch 2900, Loss: 0.187293

La curva de entrenamiento no presentó aumentos ni tampoco oscilaciones, lo que indica que el learning rate fue acertado y permitió actualizar los parámetros de manera estable. Además, la reducción del costo se volvió abrupta desde las primeras iteraciones, lo cual sugiere que el modelo se aproximó a un punto de convergencia y aumentar el número de epochs, en este punto, probablemente no produciría mejoras muy significativas.

B. Resultados de la clasificación

La regresión logística obtuvo una exactitud de 92.65% en entrenamiento, 93.57% en validación y 92.65% en prueba. La diferencia porcentual entre entrenamiento y validación fue de 0.92 puntos, mientras que entrenamiento y prueba presentaron la misma exactitud. Aunque validation obtuvo un resultado ligeramente mayor que train, esta diferencia puede ser producida por la composición randomizada de los conjuntos y

no representa un comportamiento anómalo en sí misma. La similitud entre los resultados indica que el modelo mantuvo consistencia en su desempeño al clasificar observaciones que no participaron en la optimización de sus parámetros.

V. IMPLEMENTACIÓN CON FRAMEWORK

A. Justificación del algoritmo

Para la segunda implementación con framework se seleccionó Random Forest, un algoritmo supervisado que consigue resolver problemas de clasificación mediante la combinación de varios árboles de decisión. Cada árbol se entrena a partir de una muestra aleatoria de las observaciones y considera sólo un subconjunto de las características al construir sus divisiones. La predicción final se obtiene observando la clase seleccionada por la mayoría de los árboles.

Como cada árbol puede capturar relaciones no lineales entre las variables de entrada y la variable objetivo, Random Forest se convierte en una opción muy interesante para este dataset de arroz, ya que las gráficas comparativas de características obtenidas durante el EDA muestran regiones de superposición entre las dos variedades pero también cúmulos separados en posiciones particulares. Además, a diferencia de la regresión logística entrenada con gradiente descendente, Random Forest no necesita que las características se estandaricen, pues las divisiones de los árboles no dependen de la magnitud relativa de las variables.

B. Preparación y separación de los datos

La implementación se realizó utilizando scikit-learn, específicamente RandomForestClassifier. De nuevo, la variable Class se codificó como 0 para Osmancik y 1 para Cammeo, y después, se utilizó train_test_split con una seed aleatoria y el parámetro stratify para conservar la distribución original de las clases.

Primero se tomó el 20% del dataset como conjunto de prueba, mientras que el 80% restante se dividió nuevamente, asignando 25% a validación. Así es como se obtuvo una distribución final de 60% para entrenamiento, 20% para validación y 20% para prueba. En este caso, el conjunto de entrenamiento se usó para ajustar los árboles y realizar la búsqueda de hiperparámetros, validation se usó para comprobar la capacidad de generalización de las configuraciones obtenidas, y test se usó hasta la evaluación final.

C. Métricas de evaluación

El desempeño general del clasificador se evaluó mediante accuracy, que representa la proporción de observaciones clasificadas correctamente. Se usó classification_report de scikit-learn para obtener precision, recall y F1-score para ambas clases, y finalmente, se calcularon matrices de confusión para los conjuntos de validación y prueba, con el propósito de identificar los errores de clasificación cometidos por el modelo en diferentes etapas.

D. Configuración inicial

El primer Random Forest fue entrenado utilizando los hiperparámetros por default de RandomForestClassifier, a excepción de random_state, que se estableció en 67 para hacer reproducibles los resultados.

VI. DIAGNÓSTICO DEL MODELO BASE CON FRAMEWORK

A. Resultados generales

Con este primer modelo base con framework, se obtuvo una accuracy de 100% en el conjunto de entrenamiento y de 92.65% en el conjunto de validación. Por lo tanto, la brecha de generalización entre estos conjuntos fue de aproximadamente 7.35%.

Este resultado perfecto en entrenamiento demostró que el modelo tuvo la capacidad para clasificar correctamente todas las observaciones utilizadas durante su ajuste, sin embargo, la caída observada en validación significa que una parte de ese desempeño no se mantuvo al cambiar a datos que no participaron en el entrenamiento.

Conjunto	Accuracy	Error
Entrenamiento	100%	0%
Validación	92.65%	7.35%

B. Análisis de la matriz de confusión

La matriz de confusión del conjunto de validación fue:

416	20
36	290

El modelo clasificó correctamente 416 observaciones de Osmancik y 290 observaciones de Cammeo, pero clasificó incorrectamente 20 observaciones de Osmancik como Cammeo y 36 observaciones de Cammeo como Osmancik. En total, realizó 706 predicciones correctas y 56 incorrectas contra las 762 observaciones totales de validación.

Classification report:				
	precision	recall	f1-score	support
0	0.92	0.95	0.94	436
1	0.94	0.89	0.91	326
accuracy			0.93	762
macro avg	0.93	0.92	0.92	762
weighted avg	0.93	0.93	0.93	762

El classification report muestra que Osmancik obtuvo un recall de 0.95, mientras que Cammeo obtuvo uno de 0.89. Esto quiere decir que el modelo identificó más observaciones reales de Osmancik que observaciones reales de Cammeo. En cambio, la precision fue de 0.92 para Osmancik y de 0.94 para Cammeo. Los F1 scores fueron de 0.94 y 0.91 respectivamente, y estos significan que el modelo logró un mejor equilibrio entre el recall y precision para Osmancik que para Cammeo. Como el F1 score combina ambas métricas en un solo valor, entre más se acerca un resultado al 1, esto indica un mejor desempeño general en la clasificación de esa clase.

Estos resultados demuestran que el desempeño general fue bueno en ambas clases, aunque al modelo se le dificultó más reconocer todos los granos pertenecientes a Cammeo, pues 36 observaciones de esta variedad fueron clasificadas incorrectamente como Osmancik.

C. Diagnóstico de sesgo, varianza y ajuste

El modelo base presenta un grado de sesgo bajo, pues su error de entrenamiento es cero, y no existen señales de que este presente underfitting. Sin embargo, sí se diagnostica un grado de varianza alto, pues aunque el modelo clasificó correctamente el conjunto de entrenamiento, su accuracy

disminuyó 7.35% en validación. Esta caída indica que el modelo fue sensible a las diferencias de los datos de entrenamiento y que parte de los patrones aprendidos no se generalizó a las observaciones no vistas, lo cual es señal de overfitting.

Posteriormente, y para bajar este sobreajuste, se buscaron y ajustaron los hiperparámetros relacionados con la profundidad y el crecimiento de los árboles para reducir la varianza y mejorar la capacidad de generalización.

VII. AJUSTE DE HIPERPARÁMETROS Y MEJORA DEL MODELO

A. Búsqueda de hiperparámetros y configuración seleccionada

Para reducir la diferencia entre entrenamiento y validación del modelo base, se realizó una búsqueda de hiperparámetros mediante GridSearchCV de scikit-learn. En la implementación del código, la ejecución de esta búsqueda se controla mediante la variable RUN_GRID_SEARCH. Para la corrida documentada se mantuvo activada, pero si se desea evitar repetir la búsqueda, la rama alternativa construye directamente el modelo con la mejor configuración encontrada con anterioridad.

Esta búsqueda se ejecutó con el conjunto de entrenamiento para evitar que los conjuntos de validación y prueba influyeran en la selección de la configuración. De igual forma, se llevó a cabo con la técnica de validación cruzada "StratifiedKFolds" con cinco folds, por lo que en cada iteración, el conjunto de entrenamiento se dividió en cinco partes con proporciones similares de las dos variedades de arroz. Después, el modelo se entrenó con cuatro de ellas y se evaluó con la restante, repitiendo el proceso hasta haberse evaluado con cada fold.

Hiperparámetro	Valores evaluados
n_estimators	[100, 300, 500]
max_depth	[None, 5, 10, 20]
min_samples_split	[2, 5, 10]
min_samples_leaf	[1, 2, 4]
max_features	["sqrt", "log2", 0.5]
class_weight	[None, "balanced"]

La búsqueda probó varios parámetros, como diferentes cantidades de árboles, profundidades máximas, números mínimos de observaciones necesarios para dividir un nodo y formar una hoja, cantidades de características consideradas en cada división y estrategias de ponderación de las clases. En total, se evaluaron 648 combinaciones de hiperparámetros, y debido a que cada configuración participó en cinco folds, la búsqueda requirió 3240 entrenamientos, los cuales localmente se ejecutaron en 8 minutos en promedio.

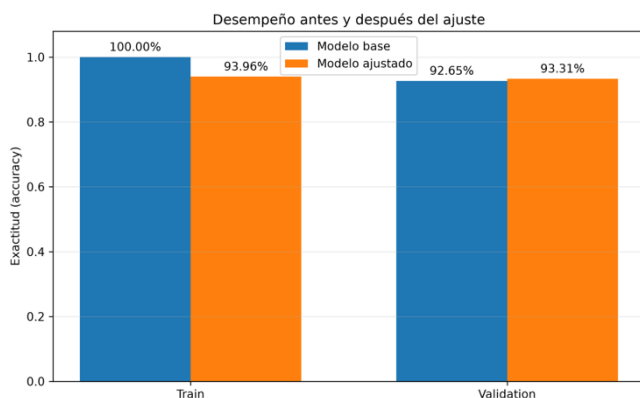
Al final, la búsqueda arrojó la siguiente configuración, la cual sería usada en la implementación final para permitir la ejecución del modelo sin tener que repetir toda la búsqueda.

Hiperparámetro	Valor
----------------	-------

n_estimators	300
max_depth	5
min_samples_split	10
min_samples_leaf	2
max_features	sqrt
class_weight	None
random_state	67

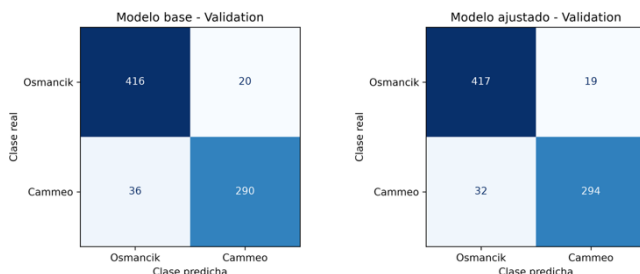
B. Comparación del desempeño

Después del ajuste, la accuracy de entrenamiento disminuyó de 100% a 93.96%, mientras que la accuracy de validación aumentó de 92.65% a 93.31%. Aunque el incremento en validación fue poco, la brecha que existía entre entrenamiento y validación se redujo de 7.35 a 0.66 puntos porcentuales, lo cual es una señal que el overfitting ha disminuido.



La matriz de confusión de validación también presentó una mejora, pues en el modelo base se clasificaron incorrectamente 20 observaciones de Osmancik como Cammeo y 36 de Cammeo como Osmancik, pero después del ajuste, estos errores disminuyeron a 19 y 32 respectivamente.

La cantidad total de errores en total se redujo de 56 a 51 observaciones y el modelo ajustado clasificó correctamente 417 de las 436 observaciones de Osmancik y 294 de las 326 observaciones de Cammeo en validation. El recall de Cammeo aumentó de 0.89 a 0.90 y su F1-score aumentó de 0.91 a 0.92.



C. Efectos del ajuste

Los resultados demuestran que el ajuste de hiperparámetros redujo la varianza del Random Forest, pues el modelo base tenía una diferencia de 7.35 puntos porcentuales entre entrenamiento y validación, mientras que el modelo ajustado la mejoró a una diferencia menor a un punto. De igual manera, la accuracy de validación se mantuvo

por encima de 93%, por lo que la regularización tampoco provocó underfitting.

VIII. EVALUACIÓN FINAL EN EL CONJUNTO DE PRUEBA

Después de seleccionar los hiperparámetros ideales mediante el conjunto de entrenamiento y comprobar su desempeño con validación, el modelo ajustado se evaluó una vez más con el conjunto de prueba. Con este conjunto obtuvo una accuracy de 92.65%, un resultado bastante cercano al 93.31% alcanzado en validación.

402	34
22	304

La matriz de confusión mostró 402 observaciones de Osmancik y 304 de Cammeo clasificadas correctamente. Por otro lado, 34 observaciones de Osmancik fueron clasificadas como Cammeo y 22 de Cammeo como Osmancik, dando un total de 706 predicciones correctas y 56 incorrectas.

Probabilidad Cammeo	Predicción	Real
0.3849	Osmancik	Osmancik
0.9880	Cammeo	Cammeo
0.0797	Osmancik	Osmancik
0.8123	Cammeo	Cammeo
0.0171	Osmancik	Osmancik
0.6741	Cammeo	Osmancik
0.8706	Cammeo	Cammeo
0.9982	Cammeo	Cammeo
0.0345	Osmancik	Osmancik
0.1081	Osmancik	Osmancik

Finalmente, se realizaron diez predicciones del conjunto de prueba junto con su probabilidad de pertenecer a Cammeo. Nueve coincidieron con su clase real y sólo una observación de Osmancik fue clasificada incorrectamente como Cammeo, con una probabilidad de 0.6741.

IX. CONCLUSIONES

En este proyecto se implementaron dos modelos muy útiles que son capaces de clasificar exitosamente las variedades de arroz Cammeo y Osmancik. De entre los dos, se experimentó y observó cómo es que el Random Forest base presentó un sesgo bajo pero varianza alta y overfitting, principalmente notable debido a su accuracy de 100% en entrenamiento y 92.65% en validación. Es aquí donde se puede notar la importancia de realizar la validación de los modelos y nunca confiar ciegamente en las cifras del entrenamiento, pues aquel es sólo uno de los pasos para desarrollar correctamente estas implementaciones. En el caso del Random Forest base, fue necesario reducir el overfitting con la técnica de GridSearch para hacer ajuste de hiperparámetros, y es gracias a eso que se consiguió que el modelo final presentara un sesgo bajo, varianza baja y un nivel de ajuste óptimo.

X. REFERENCIAS (PENDIENTE APA)

https://archive-ics-uci-edu.translate.goog/dataset/545/rice+cammeo+and+osmancik?x_tr_sl=en&x_tr_tl=es&x_tr_hl=es&x_tr_pto=tc

<https://www.geeksforgeeks.org/machine-learning/random-forest-hyperparameter-tuning-in-python/>

<https://www.ibm.com/mx-es/think/topics/random-forest>

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

<https://medium.com/@piyushkashyap045/understanding-precision-recall-and-f1-score-metrics-ea219b908093>